

- 1) Realizzate il crivello di Eratostene, un metodo per calcolare i numeri primi noto agli antichi greci. Scegliete un numero n : questo metodo calcolerà tutti i numeri primi fino a n . Come prima cosa inserite in un **Set** tutti i numeri da 2 a n . Poi, cancellate tutti i multipli di 2 (eccetto 2); vale a dire 4, 6, 8, ... Dopodiché cancellate tutti i multipli di 3 (eccetto 3), cioè 6, 9, 12, ... Arrivate fino a $n/2$, quindi visualizzate il **Set**.
 - 2) Usate una pila per invertire le parole di una frase. Continuate a leggere parole, aggiungendole alla pila, fin quando non trovate una parola che termina con un punto. A questo punto estraete tutte le parole dalla pila e visualizzatele. Realizzate la pila tramite una **Deque**.
 - 3) Scrivete un programma che, utilizzando il metodo `split` su una stringa contenente il testo di questo esercizio, determina il numero totale di parole presenti nel testo e la parola che compare con maggiore frequenza. Potreste anche pensare di utilizzare una **HashMap<String, Integer>** per memorizzare la frequenza di ciascuna parola utilizzando la parola stessa come chiave. Stampate, infine, la frequenza di ciascuna parola (è sufficiente stampare l'intera **HashMap**).
Per inserire una coppia chiave, valore nella mappa potete usare il metodo `put(K key, V value)` mentre per leggere il valore corrispondente a una chiave il metodo `V get(Object key)`.
 - 4) Scrivere un programma interattivo che, utilizzando un record *Studente* e una **Collection<Studente>** a scelta, dia la possibilità, tramite un semplice menu testuale, di scegliere fra le seguenti opzioni:
 - a. inserire un nuovo studente nella lista con dati inseriti dall'utente
 - b. cercare uno studente nella lista in base al nome e cognome
 - c. cercare uno studente nella lista in base alla matricola
 - d. cancellare uno studente dalla lista in base alla matricola
 - e. stampare l'intera lista
 - f. cancellare l'intera lista
 - g. esci
 - 5) Definite una classe **ListaCircolare** per la gestione di una lista circolare di oggetti **String**. La classe dovrà prevedere metodi per inserire un nuovo elemento alla posizione corrente, per cancellare l'elemento corrente, per leggere l'elemento corrente e per spostarsi avanti e indietro.
 - 6) Creare una classe che implementi **Collection<String>** che, nel caso si aggiungano due elementi uguali, lanci un'eccezione personalizzata (da creare anch'essa). Creare infine anche una classe di test.
 - 7) Scrivete una classe **VectorInteger** per la gestione di un array dinamico di oggetti **Integer**. La dimensione del **VectorInteger** viene definita come parametro del costruttore (di default sarà 10) e il **VectorInteger** viene inizializzato con il valore 0. L'accesso ai membri deve avvenire mediante metodi **set** e **get** che verificano le dimensioni del **VectorInteger** ed eventualmente lanciano un'eccezione. Prevedete metodi per la somma, la differenza e il prodotto scalare che restituiscono un nuovo **VectorInteger** contenente il risultato e che verificano che i **VectorInteger** su cui fare l'operazione siano della stessa dimensione (eventualmente lanciate un'eccezione). Prevedete anche un metodo che restituisce un **VectorInteger** ottenuto moltiplicandolo per uno scalare e un metodo che restituisce il modulo (double). Utilizzate un **ArrayList** per memorizzare internamente i dati. Implementate i metodi `equals` e `hashCode` l'interfaccia **Comparable<VectorInteger>** col metodo `compareTo` che restituisce 0 se i due **VectorInteger** sono uguali e 1 oppure -1 in base al confronto dei moduli. Scrivete infine un programma per testare la classe.
 - 8) In occasione di una gara canora si vuole gestire il televoto. Il pubblico da casa può votare per uno dei 15 partecipanti ma può votare al massimo 5 volte. Il sistema deve poter raccogliere i voti in forma anonima e alla fine delle operazioni stampare la classifica risultante dal televoto.
Scrivete quindi un programma che permette di inserire un nuovo voto, verifica se da quel numero telefonico sono stati effettuati meno di 5 voti e, in caso affermativo, aggiorna la classifica. Si osservi che non si possono memorizzare i singoli voti (il voto deve restare anonimo) ma bisogna memorizzare l'elenco dei votanti e del numero di voti effettuati da ciascuno.
Scrivete un programma che all'interno di un menu testuale, permette di 1) simulare l'arrivo di un nuovo voto tramite inserimento di numero di telefono del votante e numero del cantante votato, 2) stampare il totale dei voti ricevuti fino a quel momento e 3) stampare il numero di voti ricevuti da ciascun cantante.
-